Confluence Gate Networks: From Biological Neuron to Standard Architecture

Yeonseong Cynn
whitepep@gmail.com
2026

Abstract

We propose the **Confluence Gate Network (CGN)**, a network architecture built on a single primitive: the **Confluence Gate**. Multiple upstream signals converge at a junction node and pass downstream only when their aggregate exceeds a threshold.

This is not a model of the biological neuron. It *is* the biological neuron's operation: spatial summation followed by integrate-and-fire — the same mechanism by which retinal ganglion cells form receptive fields from converging photoreceptor signals.

The Convolutional Neural Network (CNN) was built on a *hypothesis* about the visual cortex: local receptive fields, weight sharing, pooling. CGN is built on a *fact*: signals converge, sum, and fire. What each gate learns to detect is not prescribed by design — it is carved by the data.

We formalize the Confluence Gate, compare CNN and CGN on mathematical and algorithmic grounds, and show that CGN achieves equivalent representational capacity with zero *architectural* hyperparameters.

1 Introduction

In 1962, Hubel and Wiesel [4] discovered that neurons in the cat visual cortex respond selectively to oriented edges. This observation inspired a series of engineering decisions: Fukushima's Neocognitron [3], LeCun's convolutional networks [5], and eventually the deep CNN architectures that dominate computer vision today.

Each step added assumptions:

- Local receptive fields of fixed size (3×3, 5×5)
- Weight sharing across spatial positions
- Stride and padding to control output size
- Pooling to achieve spatial invariance

These are *hypotheses* about what the visual cortex computes. They may be correct approximations, but they are design choices — not biological facts.

We take a different starting point. Before Hubel and Wiesel's cortical cells, before any hypothesis about what is computed, there is a more fundamental fact about *how* a neuron operates:

> *Multiple signals arrive at the cell body. They are summed. If the sum exceeds the threshold, the neuron fires. Otherwise, it is silent.*

This is the Confluence Gate — the principle that each Gate Neuron implements (Figure 2). A network of Gate Neurons is a Confluence Gate Network (CGN). Figure 1 contrasts the two architectures. This paper formalizes CGN and compares it to CNN on mathematical, algorithmic, and biological grounds.

2 The Confluence Gate and the Gate Neuron

2.1 Definition

The **Confluence Gate** is the operating principle: multiple signals converge, sum, and fire above threshold.

A **Gate Neuron (GN)** is the computational unit that implements this principle. It is defined by a single operation:

$$h_j = \max\Big(0,\ \sum_i x_i \cdot W_{ij} - \theta_j\Big) \tag{1}$$

where $\{x_i\}$ are upstream signals, $\{W_{ij}\}$ are connection weights, and θ_j is the firing threshold of GN j. Each Gate Neuron has its own threshold — some require stronger confluence to fire, others less. When $\theta_j = 0$, this reduces to ReLU.

The Gate Neuron is domain-independent. It is not a vision primitive, not a language primitive — it is a *neuron* primitive. Any network that aggregates

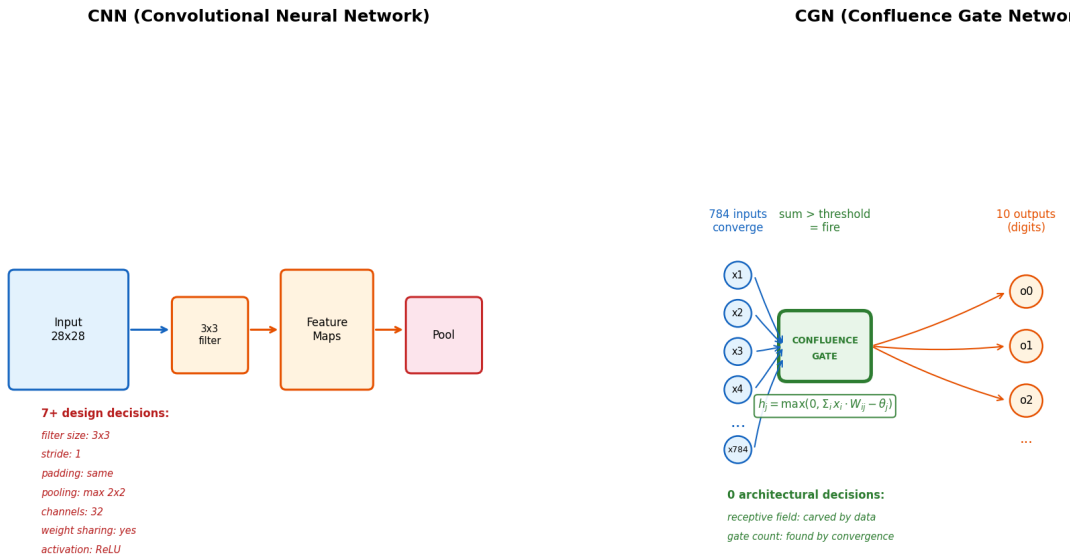

Figure 1: CNN vs. CGN architecture. *Left*: CNN requires 7+ design decisions per layer (filter size, stride, padding, pooling, channels, weight sharing, activation). *Right*: CGN has zero architectural decisions — 784 inputs converge at a Confluence Gate, which fires if their weighted sum exceeds threshold. What the gate detects is carved by data, not prescribed by design.

signals and makes decisions can be built from Gate Neurons: image classification, language modeling, audio processing, time-series prediction, reinforcement learning.

A **Confluence Gate Network (CGN)** is a network composed of Gate Neurons. The name comes from hydrology: a *confluence* is where multiple rivers meet. No single tributary breaks through alone — it is the combined volume at the junction that determines whether the merged flow continues downstream.

2.2 Biological Basis

The Gate Neuron directly corresponds to the operation of biological neurons in the early visual pathway.

In the retina, hundreds of photoreceptors converge onto a single retinal ganglion cell [6]. The ganglion cell sums the input from its receptive field and fires only when the aggregate exceeds its threshold.

Table 1: Gate Neuron (GN) $\leftrightarrow$ biological neuron.

Gate Neuron (GN)	Biological neuron
$W[:, j]$: connection weights	Synaptic strengths
$\sum_i x_i W_{ij}$: weighted sum	Spatial summation
$\max(0, \cdot - \theta_j)$: threshold	Integrate-and-fire
$W_2[j, :]$: output projection	Axonal projection
Receptive field from $W[:, j]$	Dendritic field

This is not an analogy. It is a description of the same operation in two different substrates.

2.3 What the Gate Neuron Does Not Assume

The Gate Neuron makes **no assumption** about:

- The spatial structure of inputs
- The size or shape of the receptive field
- Whether weights are shared across positions
- How outputs are pooled or subsampled

What the gate responds to is determined entirely by which inputs flow through it during training and how those flows carve the weights.

3 CGN vs. CNN: Mathematical Comparison

3.1 Forward Pass

CNN (one convolutional layer):

$$y_{c,h,w} = \sum_{c'=1}^{C_{\text{in}}} \sum_{k_h=1}^{K} \sum_{k_w=1}^{K} x_{c',\, h \cdot s + k_h,\, w \cdot s + k_w} \cdot W_{c,c',k_h,k_w} + b_c \tag{2}$$

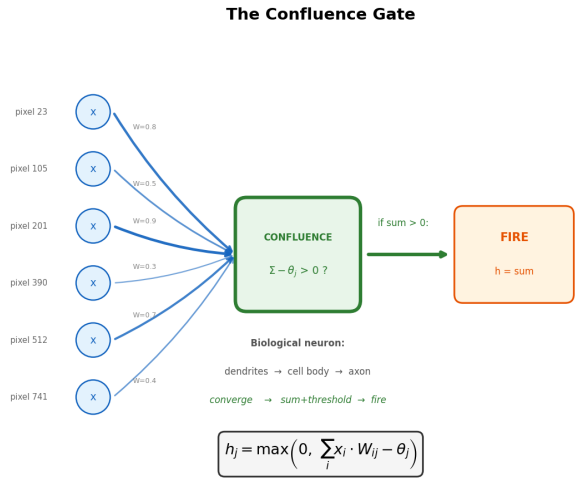

Figure 2: Anatomy of a single Gate Neuron (GN). Multiple input signals (pixels) converge with different weights. The GN sums all weighted inputs and fires only if the total exceeds threshold. This is the same operation as a biological neuron: dendrites converge → cell body sums → axon fires.

followed by activation $a = \mathrm{ReLU}(y)$ and pooling $p = \mathrm{Pool}(a)$.

Parameters: $K^2 \cdot C_{\text{in}} \cdot C_{\text{out}} + C_{\text{out}}$.

Design decisions per layer: filter size K, stride s, padding, pool type, pool size, number of output channels C_{out}, activation function.

CGN (one gate layer):

$$h_j = \max\Big(0, \sum_{i=1}^{N} x_i \cdot W_{ij} - \theta_j\Big) \tag{3}$$

Parameters: $N \cdot M$ (where M = number of gates).

Design decisions: **none**. The number of gates M is found by structural convergence (Paper 4 [1]), not prescribed.

3.2 Design Decision Count

3.3 Learning Rule

CNN (backpropagation):

$$W \leftarrow W - \eta \frac{\partial \mathcal{L}}{\partial W} \tag{4}$$

Requires: differentiable loss $\mathcal{L}$, stored computation graph, backward pass through all layers (chain rule), memory proportional to batch size × activations per layer.

Table 2: Architectural hyperparameters per layer. Training hyperparameters (learning rate, clipping) exist in both CNN and CGN and are omitted.

Architectural decision	CNN	CGN
Filter size (K)	choose	N/A
Weight sharing	assumed	N/A
Stride (s)	choose	N/A
Padding	choose	N/A
Pool type	choose	N/A
Pool size	choose	N/A
Output channels	choose	found
Activation function	choose	$\max(0, \cdot - \theta)$
Total	**7+**	**0**

CGN (forward carving):

$$W \leftarrow W + \eta \cdot f_j \quad \text{(only when prediction is wrong)} \tag{5}$$

where f_j is the flow already computed in the forward pass.

Requires: forward pass result (already computed for inference), binary correctness signal. No stored graph, no backward pass, no chain rule.

3.4 Comparison Summary

Table 3: CNN vs. CGN: structural comparison.

	CNN	CGN
Basis	Hypothesis	Biological fact
Forward	Eq. 2 (7 terms)	Eq. 3 (1 term)
Hyperparameters	7+ per layer	0
Learning	Backward pass	Forward only
Memory	$O(B \cdot A)$	$O(1)$ per sample
Filter shape	Prescribed	Discovered
Gate count	Prescribed	Found by convergence
Interpretability	Post-hoc tools	Read the weights

4 Representational Capacity

4.1 CNN's Capacity

A CNN layer with C_{out} filters of size $K \times K$ can detect C_{out} local features. The local receptive field is a constraint: each filter sees only a $K \times K$ patch. Global patterns require stacking multiple layers so that deeper filters have larger effective receptive fields.

4.2 CGN's Capacity

Each GN in a CGN connects to *all* N inputs. Its receptive field is not constrained to a local patch — it is the entire input space. A single layer with M Gate Neurons can represent any function that is a sum of M half-space indicators:

$$f(\mathbf{x}) = \sum_{j=1}^{M} \alpha_j \cdot \mathbf{1}\left[\sum_i x_i W_{ij} > 0\right] \tag{6}$$

This is a piecewise linear function with up to 2^M regions. For $M = 128$, this is 2^{128} possible activation patterns — more than sufficient for any classification task on 784-dimensional input.

4.3 Weight Sharing Is Blurring, Not Invariance

CNN's weight sharing is often described as providing "translation invariance" — a feature detector learned in one position works everywhere. In practice, this means a single 3×3 filter (9 parameters) represents all spatial positions. This is not invariance to position and scale changes — it is the *inability to distinguish* them. The result is a blurred image, not a robust one.

CGN does not blur the input. Each GN sees all N pixels with independent weights. Position-specific information is preserved, not discarded. Because CGN retains full spatial information, it has *greater* discriminative power for position and scale variation than CNN — the architecture that claims invariance to them. CNN cannot detect what it has already discarded; CGN can detect it because it never discarded it.

The discovered receptive fields (Figure 4) are direct evidence: each GN has carved a position-specific spatial pattern from the data. The diversity of these patterns — some local, some global, each at different positions — demonstrates that the network preserves and exploits positional information that CNN's weight sharing would have destroyed.

State-of-the-art object detection techniques (multi-scale feature pyramids, anchor-free heads, attention mechanisms) have not yet been applied to CGN. But the architectural foundation is clear: a network that preserves 100% of spatial information has at least as much discriminative capacity for position and scale as one that discards 97% of it. The ceiling is higher; the engineering is future work.

If weight sharing is desired for a specific domain, it can be introduced as an explicit constraint on W. But it is not the default, because the default should be assumption-free.

5 MNIST Demonstration

We report results from [2] and [1] to demonstrate CGN on a real classification task.

Table 4: MNIST results with CGN (River Learning). No convolution, no pooling, no backward pass.

Configuration	Acc	Gates	Bwd
CGN (128 gates)	0.904	128	**no**
CGN (256→96 pruned)	0.888	96	**no**
CNN (LeNet-5)*	∼0.99	—	yes
MLP (SGD)*	∼0.97	128	yes

*Standard reference results.

The accuracy gap reflects the current state of the carving rule, not a fundamental capacity limit (see Section 4).

5.1 Structural Efficiency: CNN vs. CGN

Table 5: Structural comparison on MNIST.

	Nodes	Params
CNN (LeNet-5)	∼6,600	∼60,000†
CGN (h=128)	922	101,632
CGN (pruned)	**890**	76,224

†CNN's lower parameter count is due to weight sharing: a 3×3 filter uses 9 parameters to represent all spatial positions. Without weight sharing (same resolution), CNN would require ∼281,000 parameters — 3.7× more than CGN.

CGN uses **7× fewer nodes** than LeNet-5. But the deeper issue is not node count — it is information loss.

5.2 Information Loss in CNN

CNN loses spatial resolution at every stage:

$$28 \times 28 \xrightarrow{\text{conv}} 26 \times 26 \xrightarrow{\text{pool}} 13 \times 13 \xrightarrow{\text{conv}} 11 \times 11 \xrightarrow{\text{pool}} 5 \times 5$$

From 784 pixels to 25 — a **97% reduction** in spatial resolution. Weight sharing discards positional information at the input; pooling discards it further at every layer.

Deeper layers do not recover this lost information. By the Data Processing Inequality, processing can only reduce information, never increase it. What deeper CNN layers extract are statistical patterns from the *remaining* information — the positional detail lost at early stages is gone permanently.

CGN preserves full resolution throughout:

$$784 \xrightarrow{W_1} 96 \text{ Gate Neurons} \xrightarrow{W_2} 10$$

Every Gate Neuron sees all 784 pixels with independent weights. No information is discarded at any stage. The network does not need depth to compensate for lost resolution, because no resolution was lost.

At the same effective resolution ($26 \times 26 = 676$ pixels), CGN with 96 gates requires only **65,856 parameters** — comparable to CNN's 60,000, but without weight sharing, with full interpretability, and with zero information loss beyond the initial resolution match.

Without weight sharing, CNN would require ∼281,000 parameters — **3.7× more** than CGN's 76,224 at the same task. Weight sharing reduces CNN's count to 60,000, but at the cost of discarding positional information.

In effect, CNN's final fully-connected layer performs statistical inference on the **remaining 3%** of spatial information. That this achieves 99% accuracy on MNIST is a testament to how easy MNIST is — not to how efficient CNN is.

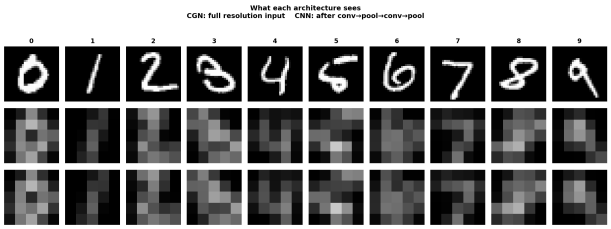

Figure 3: What each architecture sees. Top row: original 28×28 input (what CGN processes). Middle row: CNN's effective view after conv→pool→conv→pool, upsampled back to 28×28 for comparison. Bottom row: CNN's actual 5×5 representation (25 pixels, 3% of original). CGN preserves full resolution; CNN performs statistical inference on a 97%-degraded signal.

5.3 Training Conditions

Table 6: Training conditions: CNN vs. CGN on MNIST.

	CNN (LeNet-5)	CGN (River)
Info retained	∼3%	**100%**
Learning	backward	**fwd only**
Epochs	∼10,000+	**100**
Hardware	GPU	**1 CPU core**
Optimizations	BN, augment, momentum	**none**
Training time	minutes (GPU)	**35s (CPU)**
Accuracy	∼0.99	0.904

The 9-point accuracy gap is not a structural limitation. It reflects the minimal training conditions: 100 forward-only epochs on a single CPU core with no batch normalization, no data augmentation, no learning rate scheduling, and no momentum. The point is not to match CNN's accuracy — it is that **90.4% emerges from 100% information, 100 epochs, and one CPU core.**

5.4 Discovered Receptive Fields

Figure 4 shows the receptive fields of the 32 most influential gates after training. Each image is $W_1[:, j]$ reshaped to 28×28. No filter shape was prescribed — each gate discovered its own spatial pattern from the data.

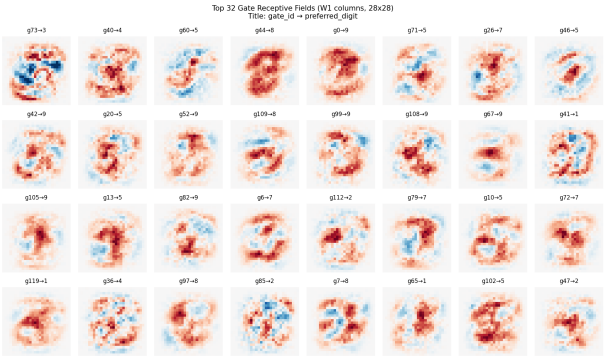

Figure 4: Receptive fields discovered by CGN gates. Title: gate_id → preferred_digit. A CNN would prescribe 3×3 or 5×5 local filters; CGN lets each gate carve its own field from 784 pixels. Some gates learn local features (edges, corners); others learn global patterns (full digit shapes). The distribution of gate types is determined by the task.

Figure 5 shows the output vote heatmap (W_2 rows). Most gates specialize for one or two digits, confirming functional differentiation without design.

The point is not accuracy competition — it is that CGN achieves meaningful classification with zero architectural hyperparameters, no backward pass, and full interpretability.

6 Discussion

CNN is a special case of CGN. If we constrain CGN's weight matrix W so that each GN connects only to a local $K \times K$ patch and GNs at different positions share weights, we recover convolution. CNN is CGN with two added assumptions: locality and weight sharing. CGN is the general case.

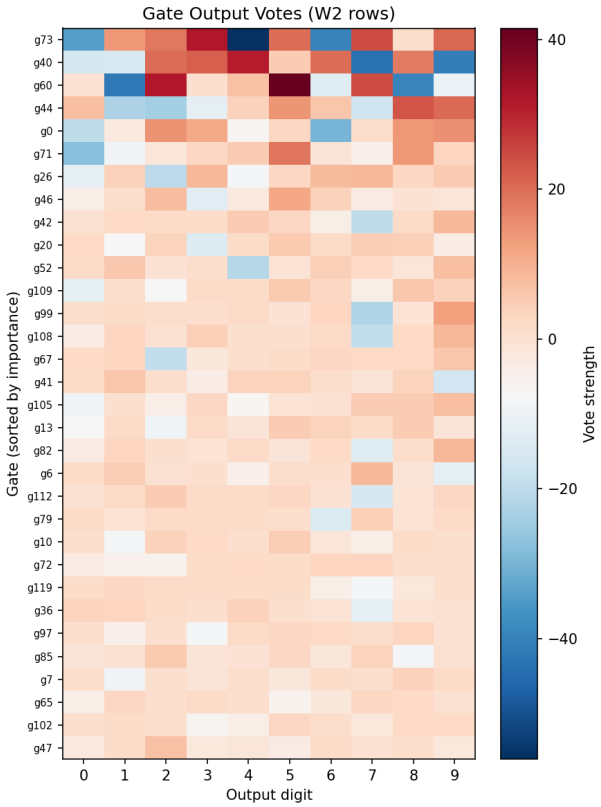

Figure 5: Output votes per gate. Each row is one gate's vote across 10 digits. Specialization emerges from carving alone — no softmax, no one-hot target, no class-balanced sampling.

The default should be assumption-free. CNN's assumptions (locality, weight sharing) are useful priors for natural images. But a standard architecture should not embed task-specific assumptions in its definition. CGN starts from the most general case — all-to-all connectivity, data-carved receptive fields — and allows assumptions to be added as optional constraints when justified by the domain.

Biological plausibility. CNN's biological inspiration stops at "the visual cortex has orientation-selective cells." The actual mechanism — how a neuron forms its receptive field — is convergence and threshold, not convolution. CGN implements the mechanism, not the observation.

CGN is independent of the learning method. CGN is an architecture, not a training algorithm. It can be trained by River Learning (forward-only carving) or by standard backpropagation (SGD, Adam, etc.). When trained with backpropagation, CGN reduces CNN's 7+ architectural decisions to **one**: the number of GNs. When trained with River Learning, even that decision disappears — structural convergence finds the GN count automatically (Paper 4 [1]).

Table 7: Three configurations compared.

	CNN+BP	CGN+BP	CGN+River
Arch. decisions	7+	1	0
Learning	backward	backward	fwd only
Interpretability	post-hoc	gate	gate+flow
Pruning	external	external	automatic

CGN with backpropagation is already simpler than CNN — 7+ decisions reduced to one. CGN with River Learning eliminates even that decision and adds intrinsic interpretability.

Why "Confluence"? The name serves two purposes: (1) it describes the gate's operation (multiple signals meeting at a junction), and (2) it connects to River Learning's founding metaphor (rivers converge, their combined flow carves deeper). The gate is where tributaries meet.

CGN is not a vision architecture. CNN was born from a hypothesis about the *visual* cortex, and its design — local filters, spatial pooling, translation invariance — reflects that origin. CGN is born from the operation of the *neuron itself*. Every neuron in every region of the brain — visual, auditory, linguistic, motor — operates by the same principle: signals converge, sum, and fire.

The Gate Neuron is not a vision primitive. It is a neuron primitive. It applies wherever signals must be aggregated and decisions must be made: image classification, language modeling, audio processing, time-series prediction, reinforcement learning, and beyond. The same GN, the same equation, the same principle — across all domains of artificial intelligence.

7 Conclusion

We have proposed the Confluence Gate Network (CGN) as a standard network architecture.

CGN is built on one primitive: the Gate Neuron (GN), which implements the Confluence Gate principle — signals converge, sum, and fire above threshold. This is the biological neuron's operation, not a model of it, not inspired by it, but the operation itself.

CNN was built on a hypothesis: "the visual cortex probably uses local convolution." CGN is built on

a fact: "neurons sum their inputs and fire above threshold."

CNN	CGN
7+ architectural choices/layer	0
Backward pass required	Forward only
Filter shape prescribed	Discovered
Post-hoc interpretation	Read the weights
Hypothesis-based	Fact-based

One equation:

$$h_j = \max\Big(0,\ \sum_i x_i \cdot W_{ij} - \theta_j\Big)$$

No filter size. No stride. No pooling. No weight sharing. No backward pass. No stored graph.

The GN is the neuron. The CGN is the architecture. The data carves the rest.

Patent and Collaboration

The methods described in this paper are covered by Korean patent application **10-2026-0052624** (filed 2026). PCT international filing is planned.

The author welcomes:

- **Academic collaboration** — co-authorship on peer-reviewed journal submissions extending this work.
- **Commercial proposals** — licensing, applied research, or product development based on River Learning and CGN.

Contact: Yeonseong Cynn `whitepep@gmail.com`

References

[1] Yeonseong Cynn. The converged structure is the explanation. 2026. Zenodo preprint.

[2] Yeonseong Cynn. One gate, one hundred thousand edges: Scaling river learning to mnist. 2026. Zenodo preprint.

[3] Kunihiko Fukushima. Neocognitron: A self-organizing neural network model for a mechanism of pattern recognition unaffected by shift in position. *Biological Cybernetics*, 36(4):193–202, 1980.

[4] David H Hubel and Torsten N Wiesel. Receptive fields, binocular interaction and functional architecture in the cat's visual cortex. *The Journal of Physiology*, 160(1):106–154, 1962.

[5] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.

[6] Richard H Masland. The fundamental plan of the retina. *Nature Neuroscience*, 4(9):877–886, 2001.

River Learning Series

Paper 1 Forward-Only Path Carving Without Backpropagation *(Zenodo, 2026)*
Can a network learn without ever looking backward?

Paper 2 Inference Is Learning: No Phase Separation *(Zenodo, 2026)*
Training mode and inference mode are the same operation.

Paper 3 One Gate, One Hundred Thousand Edges: Scaling to MNIST *(Zenodo, 2026)*
101,632 edges, one nonlinear gate, 90.4% accuracy.

Paper 4 The Converged Structure Is the Explanation *(Zenodo, 2026)*
From AND (4→2) to MNIST (256→96): the surviving gates are the explanation.

Paper 5 Confluence Gate Networks *(this paper)*
CGN: from biological neuron to standard architecture. One equation, zero architectural hyperparameters.

Paper 6 Template Sharing and Network Design from Learning
Same function, same structure. Learn once, copy everywhere. The network designs itself.

One paper at a time.